

# Non-Linear Modeling

## Chapter 7 lab handout

Last modified: 2026-09-29

This handout adapts the Chapter 7 Python lab from the *Introduction to Statistical Learning with Applications in Python* (ISLP) textbook, with permission from the source available at <https://www.statlearning.com>. The original lab is `Ch07-nonlin-lab.Rmd` in the `intro-stat-learning/ISLP_labs` repository.

Students who prefer **R** can work through the corresponding chapter of Emil Hvitfeldt's [ISLR tidymodels labs](#).

In this lab we demonstrate some of the nonlinear models discussed in this chapter. We use the `Wage` data as a running example, and show that many of the complex non-linear fitting procedures discussed can easily be implemented in Python.

## 1 Setup

As usual, we start with some of our standard imports.

```
import numpy as np, pandas as pd
from matplotlib.pyplot import subplots
import statsmodels.api as sm
from ISLP import load_data
from ISLP.models import (summarize,
                          poly,
                          ModelSpec as MS)
from statsmodels.stats.anova import anova_lm
```

We again collect the new imports needed for this lab. Many of these are developed specifically for the ISLP package.

```
from pygam import (s as s_gam,
                   l as l_gam,
                   f as f_gam,
                   LinearGAM,
                   LogisticGAM)

from ISLP.transforms import (BSpline,
                              NaturalSpline)
```

```

from ISLP.models import bs, ns
from ISLP.pygam import (approx_lam,
                        degrees_of_freedom,
                        plot as plot_gam,
                        anova as anova_gam)

```

## 2 Polynomial Regression and Step Functions

**Exercise 1.** Load the Wage data with `load_data('Wage')`. Store the response (`wage`) as `y` and the predictor (`age`) as `age`.

**Exercise 2.** Use `poly()` inside `MS()` to build a 4th-degree polynomial in `age`, fit an OLS model of `y` on this basis, and print the coefficient summary.

**Exercise 3.** Build a grid `age_grid` of 100 evenly spaced values spanning the observed range of `age`, and wrap it in a single-column `DataFrame` named `age_df`.

**Exercise 4.** Write a function `plot_wage_fit(age_df, basis, title)` that fits OLS on the supplied basis, evaluates it on `age_df`, and plots the raw data with the fitted curve and its 95% confidence bands.

**Exercise 5.** Use your helper to plot the degree-4 polynomial fit.

**Exercise 6.** With polynomial regression we must decide on the degree of the polynomial to use. Fit polynomial models in `age` of degrees 1 through 5, and use `anova_lm()` to test which degree is sufficient.

**Exercise 7.** Repeat the ANOVA, but with a linear term in `education` included in each model alongside a polynomial in `age` of degrees 1, 2, 3.

**Exercise 8.** Now consider predicting whether someone earns more than \$250,000 per year. Fit a logistic regression (use `sm.GLM` with a `Binomial` family) of `wage > 250` on the same 4th-degree polynomial basis in `age`, and produce a plot of the predicted probability vs. `age` (with confidence bands and a rug plot).

**Exercise 9.** Fit a step-function regression: discretise `age` into 4 quantile-based bins with `pd.qcut()`, build dummy columns with `pd.get_dummies()`, and fit OLS.

## 3 Splines

**Exercise 10.** Use `BSpline()` from `ISLP.transforms` with `internal_knots=[25, 40, 60]` and `intercept=True` to build a cubic B-spline basis for `age`. What shape is the resulting design matrix?

**Exercise 11.** Now fit a cubic spline regression of `wage` on `age` using the helper `bs()` inside `MS()`. Pass `name='bs(age)'` to clean up the column labels in the summary.

**Exercise 12.** Instead of specifying knots, ask `BSpline` for `df=6`. Inspect `internal_knots_` to see where the knots ended up.

**Exercise 13.** B-splines need not be cubic. Fit a degree-0 spline with `df=3` and compare the coefficients to your earlier `qcut()` step-function fit.

**Exercise 14.** Fit a natural cubic spline with 5 degrees of freedom using `ns()`, and plot the fit with `plot_wage_fit()`.

## 4 Smoothing Splines and GAMs

**Exercise 15.** A smoothing spline is a special case of a GAM with squared-error loss and a single feature. Fit `LinearGAM(s_gam(0, lam=0.6))` to predict `wage` from `age`. (Remember `pygam` expects a 2-D feature matrix.)

**Exercise 16.** Investigate how the fit changes with the smoothing parameter. Loop `lam` over `np.logspace(-2, 6, 5)` and plot each resulting fit.

**Exercise 17.** `pygam` can search for an optimal smoothing parameter. Reproduce the plot from Exercise 16, call `.gridsearch(X_age, y)`, and add the resulting fit on top.

**Exercise 18.** It's often more interpretable to specify the *effective degrees of freedom* than `lam`. Use `approx_lam()` from `ISLP.pygam` to find a `lam` giving roughly 4 degrees of freedom for the `age` term, and verify with `degrees_of_freedom()`.

**Exercise 19.** Now loop over `df` values `[1, 3, 4, 8, 15]` (passing `df + 1` to `approx_lam()` to account for the intercept), refit, and plot each.

### 4.1 Additive Models with Several Terms

**Exercise 20.** Build a GAM “by hand”: use natural splines (`NaturalSpline`) with `df=4` for `age` and `df=5` for `year`, get dummies for `education`, stack them all into a model matrix `X_bh`, and fit OLS.

**Exercise 21.** Construct a partial-dependence plot for `age` from the by-hand GAM: predict on a matrix where all columns except those for `age` are held at their column means, and the `age` columns are swept across `age_grid`.

**Exercise 22.** Do the same for `year`.

**Exercise 23.** Now fit the same GAM using *smoothing* splines via `pygam`. Build `LinearGAM(s_gam(0) + s_gam(1, n_splines=7) + f_gam(2, lam=0))`, where `f_gam` is a factor term for `education`. (Convert `education` to numeric codes with `.cat.codes`.) Then plot the partial dependence for `age` using `plot_gam()` from `ISLP.pygam`.

**Exercise 24.** Re-tune `gam_full` so that the `age` and `year` smoothers each have  $\sim 4$  degrees of freedom (use `approx_lam()` with `df=4+1`), then re-plot the partial dependence for `age` and for `year`.

**Exercise 25.** Plot the partial dependence for `education` — a categorical term gets a different style of plot, suited to a set of fitted constants per level.

### 4.2 ANOVA Tests for Additive Models

**Exercise 26.** In all our models, the function of `year` looks fairly linear. Compare three models by ANOVA: a GAM that *excludes* `year` ( $\mathcal{M}_1$ ), a GAM that uses a *linear* term in `year` ( $\mathcal{M}_2$ ), and the full GAM with a smoothing spline in `year` ( $\mathcal{M}_3$ ). What do you conclude?

**Exercise 27.** Repeat the test for `age`.

**Exercise 28.** Print `gam_full.summary()` and use `gam_full.predict()` to generate fitted values on the training set.

**Exercise 29.** Fit a logistic GAM for `high_earn` (i.e. `wage > 250`) using `LogisticGAM(age_term + l_gam(1, lam=0) + f_gam(2, lam=0))`. Plot the partial dependence for `education`.

**Exercise 30.** Cross-tabulate `high_earn` against `education`. What do you see?

**Exercise 31.** Refit the logistic GAM excluding observations in the “1. < HS Grad” category, then plot the partial dependence for `education`, `year`, and `age`.

## 5 Local Regression

**Exercise 32.** Finally, use `sm.nonparametric.lowess` to fit local linear regression models with spans of 0.2 and 0.5, and plot them together. Some implementations of GAMs allow local-regression terms; `pygam` does not.